

Optimizing a Genetic Algorithm on Benchmark Optimization Functions

Obreja Rares

December 2, 2025

Abstract

We designed a Genetic Algorithm that uses a population of candidate solutions and a selection process applied to it throughout multiple generations in order to get as close as possible towards the minimum of a few numerical benchmark functions (De Jong's (Sphere model), Rastrigin's, Schwefel's and Michalewicz's functions). Then we optimized this algorithm by using Gray Encoding for the candidate solutions, improving the fitness evaluation function and using a larger population with fewer generations. We also tried multiple versions of meta-genetic algorithms but couldn't make them give good enough results (on Michalewicz they were slightly worse than on the original Genetic Algorithm). In the end the optimized version obtained better results on all the functions we tested it on, finding the optimum for both De Jong's and Rastrigin's functions and getting very close to it for Schwefel's function (it reached an average of 0.225). On Michalewicz however, the improvement wasn't as good, only going from -28.083 to -28.478 . Surprisingly, the algorithms gave better results with a more permissive fitness function. We tried to increase the selection pressure through a small variable within the function, but during the experiments it showed that using a higher value (aprox. 1-2) gave better results than going lower (we tried multiple values between 0.1 and 0.000001). The optimizations worked really well on the first three functions and only slightly better on Michalewicz, the hardest one. To get closer to the optimum on this we might need to change the structure of the original Genetic Algorithm and then use a good meta-genetic algorithm in order to fine-tune the parameters used. Unfortunately, we didn't manage to successfully implement that in this paper.

1 Introduction

The Genetic Algorithm used is an extension of Hill Climb Best Improvement Algorithm that uses a population of solutions instead of just one and through crossover and mutation selects future generations in hope of finding the optimum solution within the population. The selection process can be optimized by modifying the way we calculate the fitness function for each candidate therefore, we can have a

more relaxed constraint so that slightly worse solutions also have a good chance of being selected, or we can be very rigid and only allow the best ones in the next generation which results in an elitism approach. Another important aspect is the number of crossovers and mutations we allow which can give a huge exploration within the population or making it remain basically unchanged after these processes. Starting from this algorithm we introduced Gray Encoding and modified the selection process through the fitness function. We also tried using a higher population with fewer generations and it proved to obtain better values over all the numerical functions we tested them on. The purpose of this paper is the comparative study of this Genetic Algorithm with the later optimized version to optimize some well-known multidimensional mathematical functions (De Jong's, Schwefel's, Rastrigin's, and Michalewicz's). The implemented algorithms are evaluated based on their convergence behavior and robustness in finding near optimal solutions within bounded search spaces.

2 Algorithms used and their implementation

2.1 Genetic Algorithm

A genetic algorithm (GA) is a computational method inspired by natural selection to find approximate solutions for optimization and search problems. It involves creating a population of potential solutions, evaluating their fitness using an objective function, and then using operators like selection, crossover, and mutation to create new, fitter generations. This process repeats until a satisfactory solution is found.

In our implementation we started with a random population that we improved by applying HC Best Improvement to each candidate so we start with a better population than just random candidates. Then we evaluated it with a fitness function so that better results (smaller values on the numerical function evaluation) had a bigger fitness in order to have a better chance of being selected for the next generation through the wheel of fortune method. After that we choose the ones that will undergo crossover. From those we select pairs of two that in return will give two offsprings. For each of the offsprings we apply HC Best Improvement in order to fine-tune them into better solutions. In the end we apply a mutation so that each bit in each candidate solution has an equal chance of being selected and we mutate it by flipping its value.

2.1.1 Implementation

- Solutions - represented as a vector or bits containing all the dimension values one after another (each value is represented on 30 bits - the whole solution has at most 900 bits in the case of 30 dimensions)
- Population Size - 100
- Crossover probability - 0.4

- Mutation probability - 0.002
- Selection pressure (in order to give better chances at being selected toward the better candidates) - $\text{best_of_gen} / 2$ (best_of_gen for Michalewicz's function also for turning the values positive)
- Fitness function - $1.0 / (f(x_i) - \text{selection_pressure} + 1)$

2.2 Optimized Genetic Algorithm

Starting from the Genetic Algorithm presented above we added gray encoding, modified the fitness function and increased the population with less generations. Also for Rastrigin's, DeJong's and Michalewicz's functions we used a smaller number of bits for the representation of the solution in order to reduce the complexity.

2.2.1 Implementation

- Solutions - represented as a vector of bits (gray encoding) containing all the dimension values one after another (each values is represented on 20 (30) bits - the whole solution has at most 600(900) bits in the case of 30 dimensions)
- Population Size - 400
- Crossover probability - 0.3
- Mutation probability - 0.002
- Selection pressure (in order to give better chances at being selected toward the better candidates) - best_of_gen
- eps - 2 (we tried with multiple values from $1\text{E-}6$ to 15 and it gave better results with this value)
- Fitness function - $1.0 / (f(x_i) - \text{selection_pressure} + \text{eps})$

3 Functions Used

3.1 De Jong (sphere model)

De Jong's function is continuous, convex and uni-modal with a global minimum of 0 that is reached when the value for each dimension is also 0.

Function definition:

$$f(x) = \sum_1^n (i \cdot x_i^2), \quad -5.12 \leq x_i \leq 5.12.$$

3.2 Rastrigin

Rastrigin's function is highly multimodal. However, the location of the minima are regularly distributed with the global minimum value being 0 that is reached when the value for each dimension is also 0, just as De Jong's function.

Function definition:

$$f(x) = 10 \cdot n + \sum_1^n (x_i^2 - 10 \cdot \cos(2 \cdot \pi \cdot x_i)), \quad -5.12 \leq x_i \leq 5.12.$$

3.3 Schwefel

Schwefel's function is deceptive in that the global minimum is geometrically distant, over the parameter space, from the next best local minima. Therefore, the search algorithms are potentially prone to convergence in the wrong direction. On our test we used the version that has 0 as the global minimum.

Function definition:

$$f(x) = \sum_1^n (-x_i \cdot \sin(\sqrt{|x_i|})) + n \cdot 418.9829, \quad -500 \leq x_i \leq 500.$$

3.4 Michalewicz

Michalewicz's function is a multimodal test function ($n!$ local optima). The parameter m defines the "steepness" of the valleys or edges. Larger m leads to more difficult search. For very large m the function behaves like a needle in the haystack (the function values for points in the space outside the narrow peaks give very little information on the location of the global optimum).

Function definition:

$$f(x) = - \sum_1^n \sin(x_i) \cdot \left(\sin \left(\frac{i \cdot x_i^2}{\pi} \right) \right)^{2 \cdot m}, \quad 0 \leq x_i \leq \pi$$

4 Experimental Results

For the Genetic Algorithm we started with a population of 100, while on the optimized version we used a population of 400 with fewer generations on multiple dimensions (5, 10, 30) at least 30 times to be able to draw a robust conclusion out of the results.

The best improvements on the Genetic Algorithm were:

- Adding Selection Pressure to the fitness function
- Applying HC Best Improvement to the initial population
- Applying HC Best Improvement to the offsprings that result from crossover.
- Other optimization is recalculating only the dimension where the bit was flipped to generate the neighbor instead of calculating all the dimensions again

The best improvements on the Optimized Genetic Algorithm were:

- Adding Gray encoding
- Modifying the Selection Pressure to the fitness function with a new parameter ϵ
- Using a larger population with a small number of generations
- Using a slightly smaller probability for crossover (from 0.4 to 0.3)
- Using fewer bits for a solutions in the case of De Jong's, Rastrigin's and Michalewicz functions

4.1 De Jong

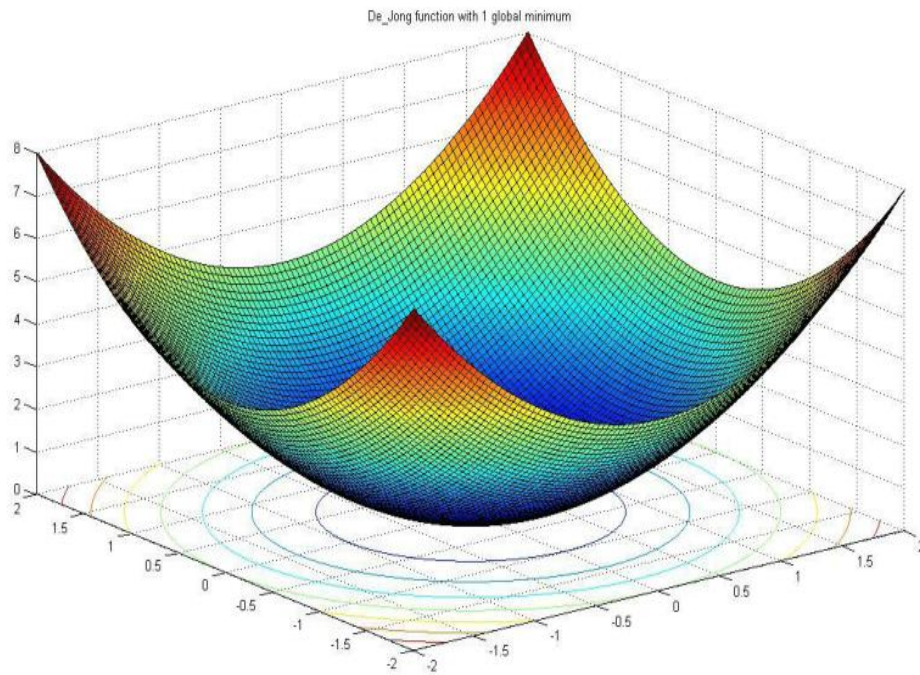


Figure 1: De Jong Sphere Model [5].Ali R. Alroomi

		GA	Optimized GA
n=30	Min	0	0
	Max	0	0
	Mean	0	0
	StdDev	0	0
	AvgRunTime(ms)	361483	401661
n=10	Min	0	0
	Max	0	0
	Mean	0	0
	StdDev	0	0
	AvgRunTime(ms)	61467	128525
n=5	Min	0	0
	Max	0	0
	Mean	0	0
	StdDev	0	0
	AvgRunTime(ms)	22309	55070

On De Jong's function 1 (sphere model) we can observe that both the algorithms reach the optimum with a similar runtime for 30 dimensions (slightly bigger for the optimized version), but as the number of dimensions get smaller the optimized version has a runtime of more than double of the original Genetic Algorithm.

4.2 Rastrigin

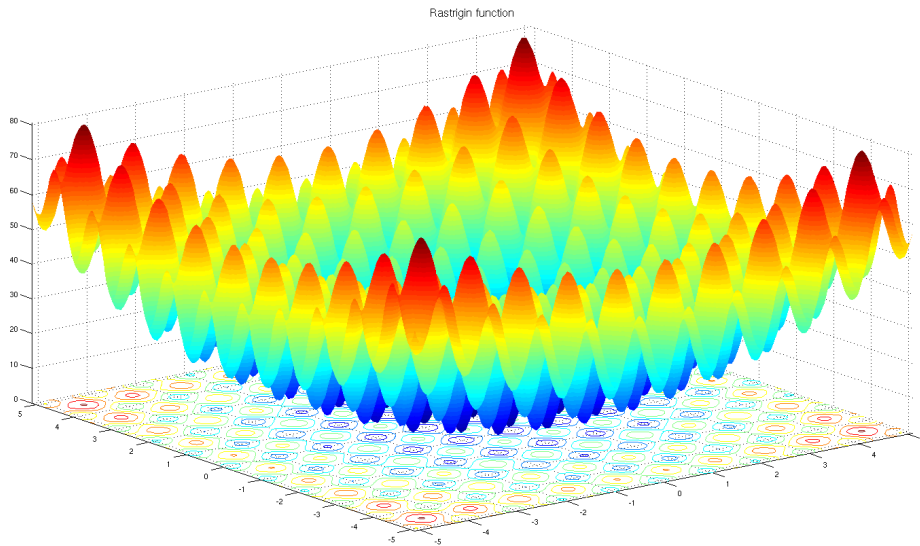
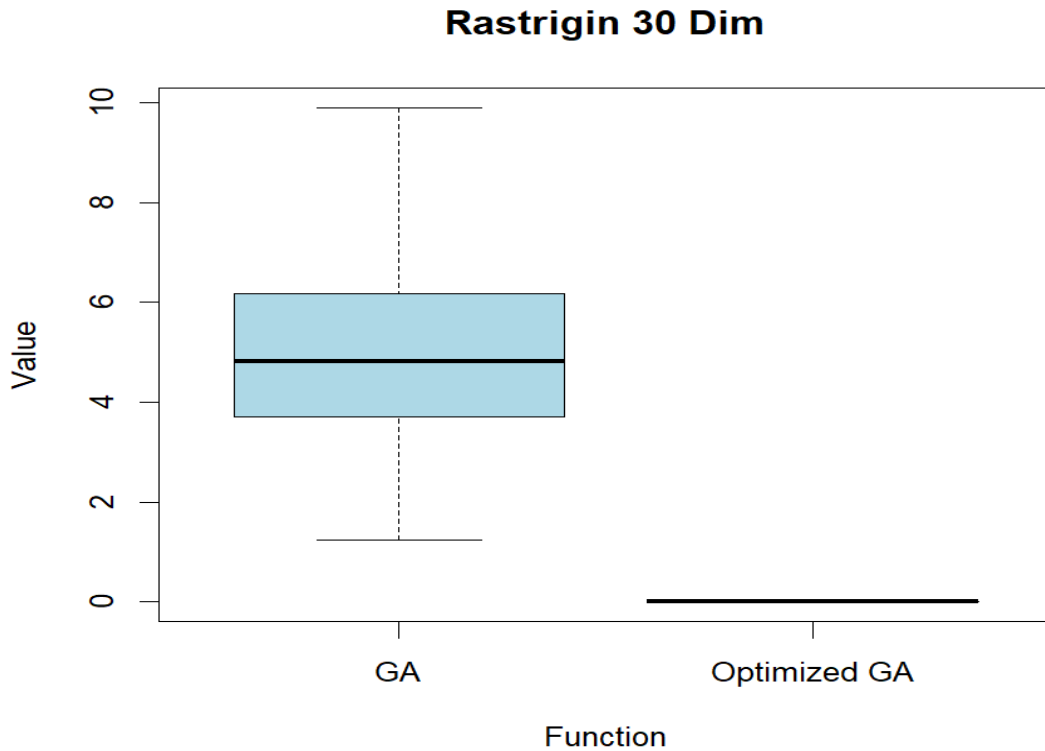


Figure 2: Schwefel's Function [4].Diegotorquemada

		GA	Optimized GA
n=30	Min	1.23576	0
	Max	9.88614	0
	Mean	4.76241	0
	StdDev	2.0345	0
	AvgRunTime(ms)	341928	415785
n=10	Min	0	0
	Max	2.47153	0
	Mean	0.17163	0
	StdDev	0.52452	0
	AvgRunTime(ms)	51803	129373
n=5	Min	0	0
	Max	1.23576	0
	Mean	0.03432	0
	StdDev	0.20596	0
	AvgRunTime(ms)	18961	48473



On Rastrigin's function we can see that the optimized version of the GA finds the optimum solution on all the dimensions, while the original version gets close. The runtime on the optimized version is also larger on all dimensions and the difference between the two gets bigger as the number of dimensions lowers just like in De Jong's function.

4.3 Schwefel

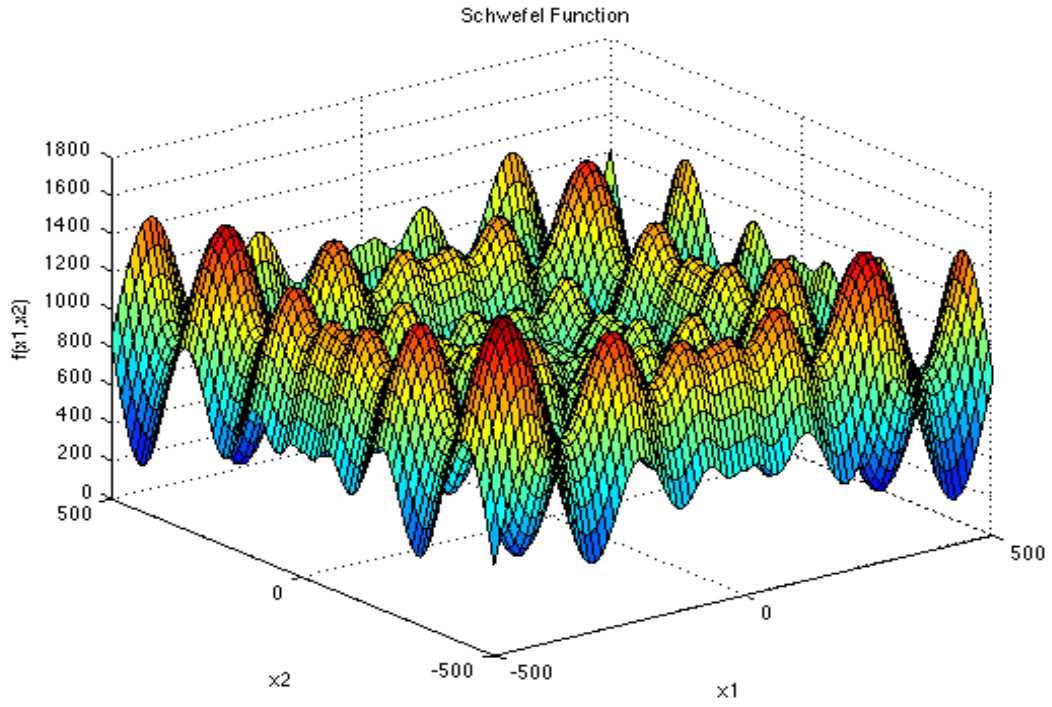
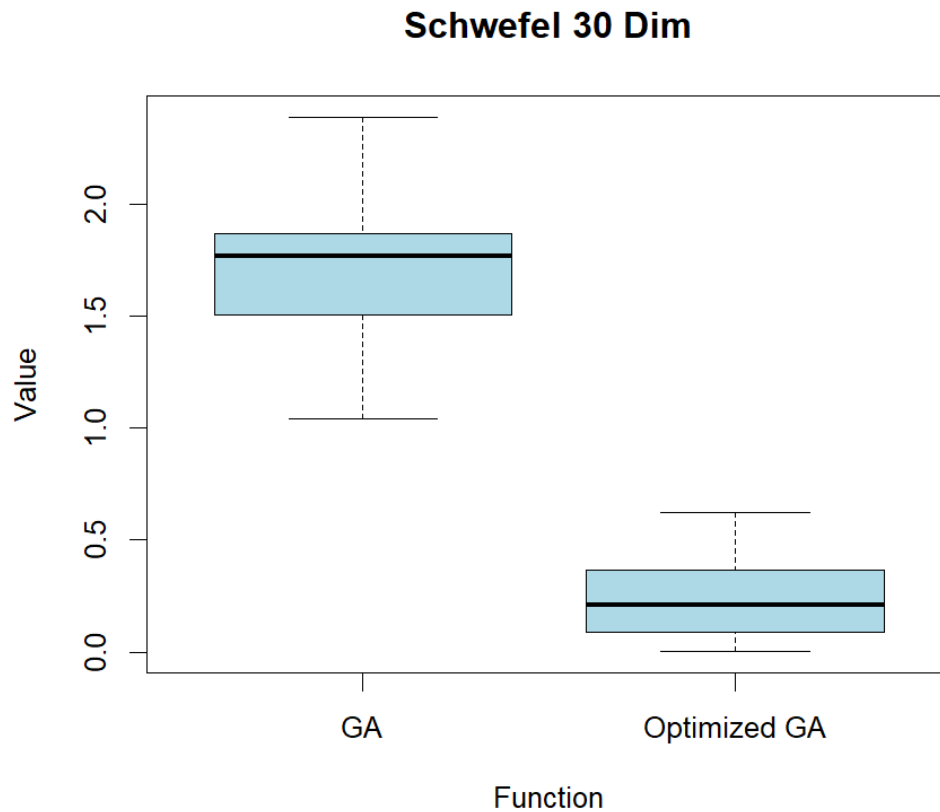


Figure 3: Schwefel's Function [6].Derek Bingham

		GA	Optimized GA
n=30	Min	1.04283	0.00038
	Max	2.3866	0.62359
	Mean	1.74912	0.22548
	StdDev	0.29184	0.18353
	AvgRunTime(ms)	374680	368014
n=10	Min	0.10506	0.00012
	Max	0.72643	0.00012
	Mean	0.47925	0.00012
	StdDev	0.16643	0
	AvgRunTime(ms)	55089	107310
n=5	Min	0.00006	0.00006
	Max	0.41473	0.00006
	Mean	0.15238	0.00006
	StdDev	0.11306	0
	AvgRunTime(ms)	19420	41379



On Schwefel we can observe that even though the optimized version doesn't reach the optimum it gets much closer to it obtaining all the values below 1. It also has a better convergence with the results being closer together. Here we can observe that the runtime for the optimized version is slightly smaller for the optimized version on 30 dimensions but it still gets much larger on fewer dimensions.

4.4 Michalewicz

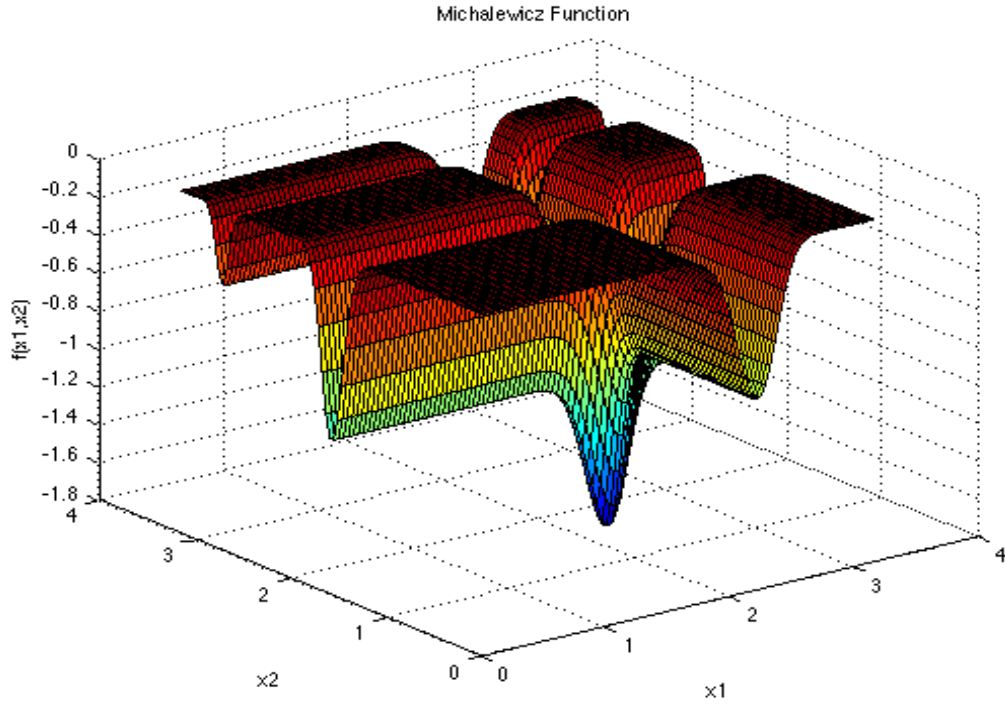
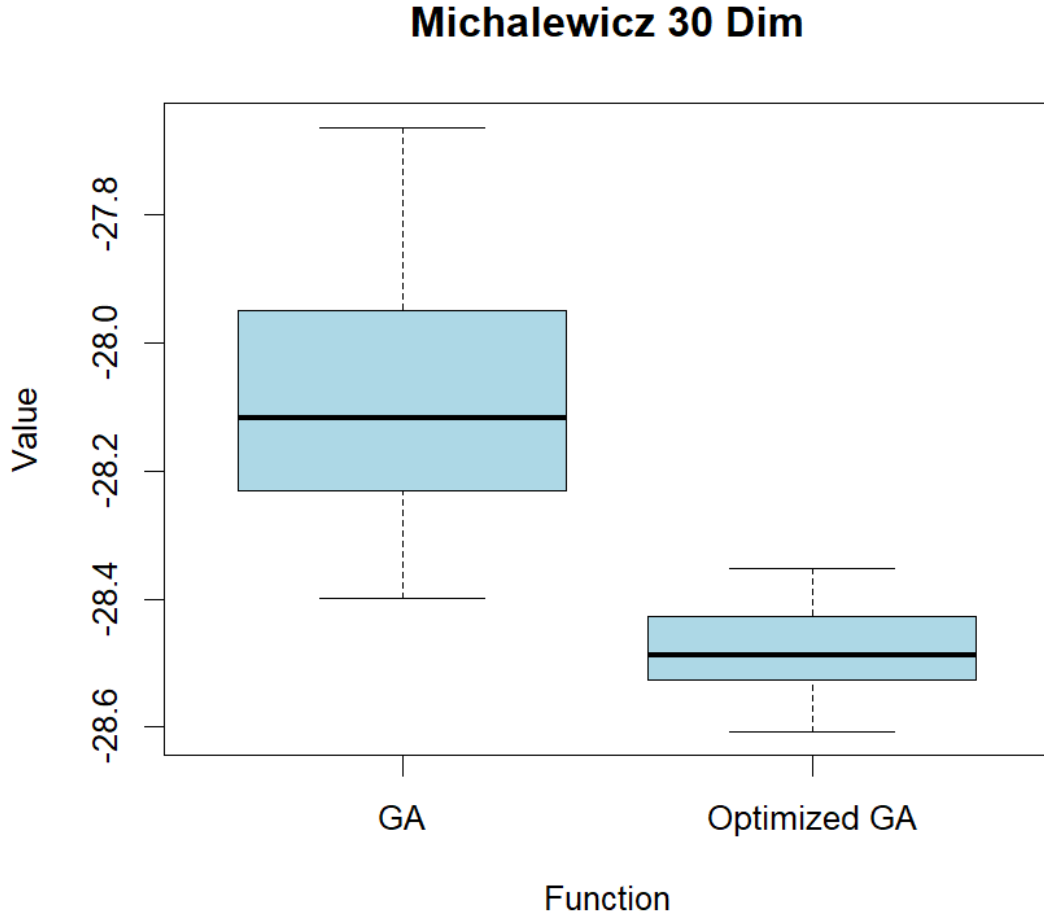


Figure 4: Michalewicz's Function [3].Derek Bingham

		GA	Optimized GA
n=30	Min	-28.39978	-28.6067
	Max	-27.66444	-28.3518
	Mean	-28.0836	-28.47841
	StdDev	0.1802	0.07458
	AvgRunTime(ms)	400724	461122
n=10	Min	-8.65945	-8.66015
	Max	-8.51013	-8.6584
	Mean	-8.63123	-8.6601
	StdDev	0.03608	0.00029
	AvgRunTime(ms)	65933	138732
n=5	Min	-3.69885	-3.69885
	Max	-3.69804	-3.69885
	Mean	-3.69881	-3.69885
	StdDev	0.00016	0
	AvgRunTime(ms)	24403	51680



On Michalewicz's function we can observe that the algorithms have closer values than on the other functions used, but with a better convergence for the optimized version. Here we can see that on the hardest function out of the four the optimized version obtains the smallest improvement and doesn't get as close to the optimum as we hoped.

5 Conclusions

In conclusion, on every function tested we obtained better results from the optimized version of the GA. They tend to have a better convergence with a less scattered distribution, however almost all the time the runtime was longer (the only exception was Schwefel 30 dimensions but the difference was small). This experiment showed that on these functions it was better to use a larger population with a smaller number of generations than a small population with a large number of generations.

Overall the optimized version performed really well on the numerical functions used. However we didn't manage to get the results we were hoping for on Michalewicz's

function. In order to get even better results we might need to use modify the original Genetic Algorithm and then optimize it with a meta-genetic algorithm. Even though we tried to implement it we couldn't make them obtain better results.

References

- [1] Eugen Croitoru, *Genetic Algorithms*, Site: <https://profs.info.uaic.ro/eugen.croitoru/teaching/ga/>.
- [2] *GEATbx: Example functions*, Site:http://www.geatbx.com/docu/fcnindex-01.html#P140_6155.
- [3] *Michalewicz Function*, Site:<https://www.sfu.ca/~ssurjano/michal.html>.
- [4] *Rastrigin Function*, Site:https://en.wikipedia.org/wiki/Rastrigin_function.
- [5] *De Jong Sphere Model Function*, Site:<https://www.al-roomi.org/component/tags/tag/1st-de-jong-s-function>.
- [6] *Schwefel's Function*, Site:<https://www.sfu.ca/~ssurjano/schwef.html>.
- [7] *Hill Climbing in Artificial Intelligence*, Site:<https://www.geeksforgeeks.org/artificial-intelligence/introduction-hill-climbing-artificial-intelligence/>.
- [8] *What is Simulated Annealing*, Site:<https://www.geeksforgeeks.org/artificial-intelligence/what-is-simulated-annealing/>.
- [9] Chinnasamy, Sathiyaraj and Ramachandran, Amudha, Ramu, Kurinjimalar, *A review on hill climbing optimization methodology*, Recent Trends in Management and Commerce, vol.3, 2022.
- [10] Goffe, William L and Ferrier, Gary D and Rogers, John, *Global optimization of statistical functions with simulated annealing*, Journal of econometrics, vol.60, 1994.
- [11] Selman, Bart and Gomes, Carla P, *Hill-climbing search*, Encyclopedia of cognitive science, vol.81, 2006.
- [12] Charlie Vanaret, Jean-Baptiste Gotteland, Nicolas Durand, Jean-Marc Alliot *Certified Global Minima for a Benchmark of Difficult Optimization Problems*, 2014. Site:https://www.researchgate.net/publication/278769271_Certified_Global_Minima_for_a_Benchmark_of_Difficult_Optimization_Problemsz.
- [13] Pablo Moscato, Carlos Cotta *A Gentle Introduction to Memetic Algorithms(2003)*. Site:https://www.researchgate.net/publication/313051410_A_Gentle_Introduction_to_Memetic_Algorithms.